
Food Ordering System

Object Constraint Language (OCL) Constraints

Software Engineering 2 - Microservices Project Documentation

This document defines the formal OCL constraints for the Food Ordering System. The constraints describe the valid business rules for users, restaurants, menu items, orders, payments, carts, and deliveries. These rules match the validation logic implemented across the backend services and support the OCL grading criterion in the project cover sheet.

Covered Area	Main Constraint Purpose
User	Valid roles, required identity fields, unique email
Restaurant and MenuItem	Restaurant ownership, required item data, positive item price
Order and OrderItem	Valid order status, positive quantity, same restaurant items
Payment	Payment method and payment status consistency
Delivery	Valid assignment and delivery status transitions
Cart	One restaurant per cart and positive item quantities

1. User Constraints

UserNameRequired

```
context User
inv UserNameRequired:
self.name <> null and self.name <> ''
```

UserEmailRequired

```
context User
inv UserEmailRequired:
self.email <> null and self.email <> ''
```

UserEmailMustBeUnique

```
context User
inv UserEmailMustBeUnique:
User.allInstances()->isUnique(email)
```

UserRoleMustBeValid

```
context User
inv UserRoleMustBeValid:
self.role = Role::CUSTOMER or
self.role = Role::ADMIN or
self.role = Role::DELIVERY
```

UserPasswordRequired

```
context User
inv UserPasswordRequired:
self.password <> null and self.password <> ''
```

2. Restaurant Constraints

RestaurantNameRequired

```
context Restaurant
inv RestaurantNameRequired:
self.name <> null and self.name <> ''
```

RestaurantAddressRequired

```
context Restaurant
inv RestaurantAddressRequired:
self.address <> null and self.address <> ''
```

RestaurantMustHaveOwner

```
context Restaurant
inv RestaurantMustHaveOwner:
self.ownerId <> null
```

RestaurantCanHaveManyMenuItems

```
context Restaurant
inv RestaurantCanHaveManyMenuItems:
self.menuItems->size() >= 0
```

3. Menu Item Constraints

MenuItemNameRequired

```
context MenuItem
inv MenuItemNameRequired:
self.name <> null and self.name <> ''
```

MenuItemPriceMustBePositive

```
context MenuItem
inv MenuItemPriceMustBePositive:
self.price > 0
```

MenuItemMustBelongToRestaurant

```
context MenuItem
inv MenuItemMustBelongToRestaurant:
self.restaurant <> null
```

MenuItemAvailabilityMustBeBoolean

```
context MenuItem
inv MenuItemAvailabilityMustBeBoolean:
self.available = true or self.available = false
```

4. Order Constraints

OrderMustHaveCustomer

```
context Order
inv OrderMustHaveCustomer:
self.customerId <> null
```

OrderMustHaveRestaurant

```
context Order
inv OrderMustHaveRestaurant:
self.restaurantId <> null
```

OrderMustHaveAtLeastOneItem

```
context Order
inv OrderMustHaveAtLeastOneItem:
self.items->size() >= 1
```

OrderTotalPriceMustBePositive

```
context Order
inv OrderTotalPriceMustBePositive:
self.totalPrice >= 0
```

OrderStatusMustBeValid

```
context Order
inv OrderStatusMustBeValid:
self.orderStatus = OrderStatus::PENDING or
self.orderStatus = OrderStatus::CONFIRMED or
self.orderStatus = OrderStatus::PREPARING or
self.orderStatus = OrderStatus::OUT_FOR_DELIVERY or
self.orderStatus = OrderStatus::DELIVERED or
self.orderStatus = OrderStatus::CANCELLED
```

DeliveredOrderCannotBeCancelled

```
context Order
inv DeliveredOrderCannotBeCancelled:
self.orderStatus = OrderStatus::DELIVERED implies
self.orderStatus <> OrderStatus::CANCELLED
```

CancelledOrderCannotBeDelivered

```
context Order
inv CancelledOrderCannotBeDelivered:
self.orderStatus = OrderStatus::CANCELLED implies
self.orderStatus <> OrderStatus::DELIVERED
```

AllOrderItemsFromSameRestaurant

```
context Order
inv AllOrderItemsFromSameRestaurant:
self.items->forall(item | item.restaurantId = self.restaurantId)
```

5. Order Item Constraints

OrderItemMustHaveMenuItem

```
context OrderItem
inv OrderItemMustHaveMenuItem:
self.menuItemId <> null
```

OrderItemQuantityMustBePositive

```
context OrderItem
inv OrderItemQuantityMustBePositive:
self.quantity >= 1
```

OrderItemPriceMustBePositive

```
context OrderItem
inv OrderItemPriceMustBePositive:
self.price > 0
```

OrderItemSubtotalMustBeCorrect

```
context OrderItem
inv OrderItemSubtotalMustBeCorrect:
self.subtotal = self.price * self.quantity
```

6. Payment Constraints

PaymentMustBelongToOrder

```
context Payment
inv PaymentMustBelongToOrder:
self.orderId <> null
```

PaymentAmountMustBePositive

```
context Payment
inv PaymentAmountMustBePositive:
self.amount >= 0
```

PaymentMethodMustBeValid

```
context Payment
inv PaymentMethodMustBeValid:
self.paymentMethod = PaymentMethod::CASH_ON_DELIVERY or
self.paymentMethod = PaymentMethod::VISA
```

CashPaymentMustBePending

```
context Payment
inv CashPaymentMustBePending:
self.paymentMethod = PaymentMethod::CASH_ON_DELIVERY implies
self.paymentStatus = PaymentStatus::PENDING
```

VisaPaymentMustBeConfirmed

```
context Payment
inv VisaPaymentMustBeConfirmed:
self.paymentMethod = PaymentMethod::VISA implies
self.paymentStatus = PaymentStatus::CONFIRMED
```

PaymentStatusMustBeValid

```
context Payment
inv PaymentStatusMustBeValid:
self.paymentStatus = PaymentStatus::PENDING or
self.paymentStatus = PaymentStatus::CONFIRMED or
self.paymentStatus = PaymentStatus::FAILED
```

7. Delivery Constraints

DeliveryMustBelongToOrder

```
context Delivery
inv DeliveryMustBelongToOrder:
self.orderId <> null
```

DeliveryMustHaveDeliveryUser

```
context Delivery
inv DeliveryMustHaveDeliveryUser:
self.deliveryUserId <> null
```

DeliveryUserNameRequired

```
context Delivery
inv DeliveryUserNameRequired:
self.deliveryUserName <> null and self.deliveryUserName <> ''
```

DeliveryStatusMustBeValid

```
context Delivery
inv DeliveryStatusMustBeValid:
self.deliveryStatus = DeliveryStatus::ASSIGNED or
self.deliveryStatus = DeliveryStatus::PICKED_UP or
self.deliveryStatus = DeliveryStatus::OUT_FOR_DELIVERY or
self.deliveryStatus = DeliveryStatus::DELIVERED or
self.deliveryStatus = DeliveryStatus::FAILED
```

DeliveredDeliveryCannotBeUpdated

```
context Delivery
inv DeliveredDeliveryCannotBeUpdated:
self.deliveryStatus = DeliveryStatus::DELIVERED implies
self.updatedAt >= self.assignedAt
```

FailedDeliveryCannotBeDelivered

```
context Delivery
inv FailedDeliveryCannotBeDelivered:
self.deliveryStatus = DeliveryStatus::FAILED implies
self.deliveryStatus <> DeliveryStatus::DELIVERED
```

DeliveryAssignedDateRequired

```
context Delivery
inv DeliveryAssignedDateRequired:
self.assignedAt <> null
```

8. Delivery Status Transition Constraints

AssignedCanMoveToPickedUpOrOutForDeliveryOrFailed

```
context Delivery::updateDeliveryStatus(newStatus: DeliveryStatus)
pre AssignedCanMoveToPickedUpOrOutForDeliveryOrFailed:
self.deliveryStatus = DeliveryStatus::ASSIGNED implies
(
  newStatus = DeliveryStatus::PICKED_UP or
  newStatus = DeliveryStatus::OUT_FOR_DELIVERY or
  newStatus = DeliveryStatus::FAILED
)
```

PickedUpCanMoveToOutForDeliveryOrFailed

```
context Delivery::updateDeliveryStatus(newStatus: DeliveryStatus)
pre PickedUpCanMoveToOutForDeliveryOrFailed:
self.deliveryStatus = DeliveryStatus::PICKED_UP implies
(
  newStatus = DeliveryStatus::OUT_FOR_DELIVERY or
  newStatus = DeliveryStatus::FAILED
)
```

OutForDeliveryCanMoveToDeliveredOrFailed

```
context Delivery::updateDeliveryStatus(newStatus: DeliveryStatus)
pre OutForDeliveryCanMoveToDeliveredOrFailed:
self.deliveryStatus = DeliveryStatus::OUT_FOR_DELIVERY implies
(
  newStatus = DeliveryStatus::DELIVERED or
  newStatus = DeliveryStatus::FAILED
)
```

FinalDeliveryStatusCannotBeChanged

```
context Delivery::updateDeliveryStatus(newStatus: DeliveryStatus)
pre FinalDeliveryStatusCannotBeChanged:
self.deliveryStatus = DeliveryStatus::DELIVERED or
self.deliveryStatus = DeliveryStatus::FAILED
implies
newStatus = self.deliveryStatus
```

9. Order Status Transition Constraints

PendingCanMoveToConfirmedOrCancelled

```
context Order::updateOrderStatus(newStatus: OrderStatus)
pre PendingCanMoveToConfirmedOrCancelled:
self.orderStatus = OrderStatus::PENDING implies
(
  newStatus = OrderStatus::CONFIRMED or
  newStatus = OrderStatus::CANCELLED
)
```

ConfirmedCanMoveToPreparingOrCancelled

```
context Order::updateOrderStatus(newStatus: OrderStatus)
pre ConfirmedCanMoveToPreparingOrCancelled:
self.orderStatus = OrderStatus::CONFIRMED implies
(
  newStatus = OrderStatus::PREPARING or
  newStatus = OrderStatus::CANCELLED
)
```

PreparingCanMoveToOutForDeliveryOrCancelled

```
context Order::updateOrderStatus(newStatus: OrderStatus)
pre PreparingCanMoveToOutForDeliveryOrCancelled:
self.orderStatus = OrderStatus::PREPARING implies
(
  newStatus = OrderStatus::OUT_FOR_DELIVERY or
  newStatus = OrderStatus::CANCELLED
)
```

OutForDeliveryCanMoveToDelivered

```
context Order::updateOrderStatus(newStatus: OrderStatus)
pre OutForDeliveryCanMoveToDelivered:
self.orderStatus = OrderStatus::OUT_FOR_DELIVERY implies
newStatus = OrderStatus::DELIVERED
```

FinalOrderStatusCannotBeChanged

```
context Order::updateOrderStatus(newStatus: OrderStatus)
pre FinalOrderStatusCannotBeChanged:
self.orderStatus = OrderStatus::DELIVERED or
self.orderStatus = OrderStatus::CANCELLED
implies
newStatus = self.orderStatus
```

10. Cart Constraints

CartMustBelongToCustomer

```
context Cart
inv CartMustBelongToCustomer:
self.customerId <> null
```

CartItemsMustBeFromSameRestaurant

```
context Cart
inv CartItemsMustBeFromSameRestaurant:
self.items->forAll(item1, item2 | item1.restaurantId = item2.restaurantId)
```

CartItemQuantityMustBePositive

```
context CartItem
inv CartItemQuantityMustBePositive:
self.quantity >= 1
```

Presentation Script

Use this short explanation during the discussion:

We used OCL to formally describe the main constraints of the Food Ordering System. The OCL rules define valid user roles, positive prices and quantities, valid order and delivery statuses, valid payment behavior, and controlled status transitions. These constraints match the business validation implemented in the backend services, such as preventing invalid delivery transitions and ensuring each order contains valid items from the same restaurant.

Checklist

- User role and identity constraints are defined.
- Restaurant and menu item constraints are defined.
- Order, order item, and cart constraints are defined.
- Payment method and payment status constraints are defined.
- Delivery assignment and delivery transition constraints are defined.
- OCL rules are aligned with implemented backend validation.